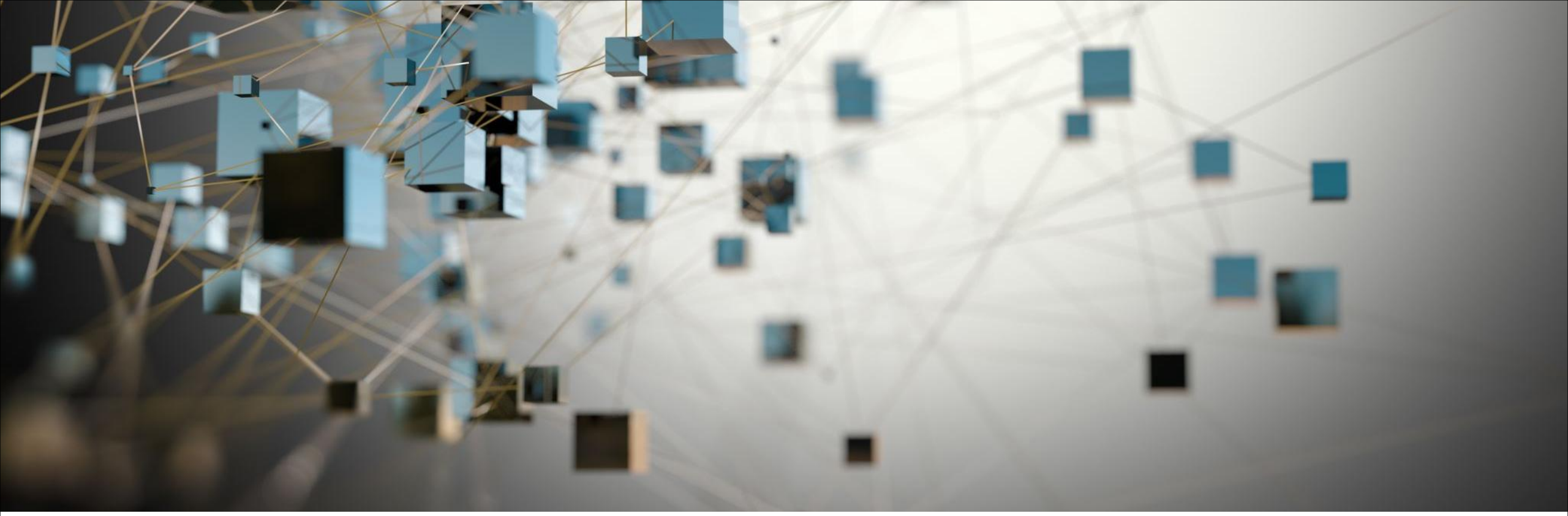


Lógica Computacional

RECURSIVIDADE

Prof. Daniel Souza

Objetivos de aprendizagem

Ao final desta aula, o aluno deverá ser capaz de:

- Compreender o fundamento matemático de funções recursivas
- Implementar algoritmos recursivos;
- Diferenciar algoritmos recursivos de iterativos;
- Identificar os pós e contras entre algoritmos recursivos e iterativos.

Recursividade

Recursividade

- Recursão é um fundamento matemático que **permite que uma função chame a si mesma para solucionar um problema**.
- Ela é utilizada quando um problema pode ser dividido em subproblemas menores e mais simples, que podem ser resolvidos utilizando a mesma função.

Motivação

A recursão é uma técnica poderosa para lidar com problemas complexos, pois ***permite expressar soluções de forma mais clara e concisa***.

Além disso, ela é muito útil para lidar com estruturas de dados complexas de forma recursiva: como ***árvores***, ***grafos*** e ***listas encadeadas***.

Definição recursiva

Definição Recursiva

Composta por:

um ***passo recursivo***:

- Estabelece uma ***relação de recorrência*** entre o problema atual e o problema anterior.
- É a chamada da própria função com argumentos diferentes, que se aproximam do caso base.

um ou mais ***casos base*** (***critérios de parada***): é a condição em que a função não precisa mais chamar a si mesma.

Definição Recursiva

Um problema p(n) pode ser resolvido a partir da recorrência da solução do problema p(n-1), mais algum tipo de operação adicional.

$$p(n) = \begin{cases} 0 & \text{se } n = 0 \text{ (caso base)} \\ p(n-1) + 1 & \text{se } n > 0 \text{ (passo recursivo)} \end{cases}$$

Um algoritmo recursivo deve

- ter um ***caso base***
- mudar o seu estado e se aproximar do caso base.
- chamar a si mesmo, recursivamente (relação de recorrência).

- $P(n) = \begin{cases} 0 & , \text{se } n = 0 \\ P(n-1) + 1 & , \text{se } n > 0 \end{cases}$

$P(n)$	Sequência	Relação
$P(0)$	0	___ CB/CP $n=0$
$P(1)$	$0 + 1 = 1$	$P(1) = P(0) + 1$
$P(2)$	$0 + 1 + 1 = 2$	$P(2) = P(1) + 1$
$P(3)$	$0 + 1 + 1 + 1 = 3$	$P(3) = P(2) + 1$
[...]		
$P(n)$	$0 + 1 + 1 + 1 + \ldots + 1 = n$	$P(n) = P(n-1) + 1$

$\boxed{\forall n > 0 \text{ Rel. Rec}}$

Pensamento recursivo

O que é necessário para que você possa construir um prédio de 3 andares?

Pense nos seguintes elementos:
- relação de recorrência com o problema atual P(3) e o problema menor P(2).
- Critério de parada (caso basse)

Construção de um prédio de 3 andares

Para construir o prédio, precisamos primeiro construir o primeiro andar, depois o segundo andar em cima do primeiro, e por fim o terceiro andar em cima do segundo. Podemos expressar esse processo recursivamente como:

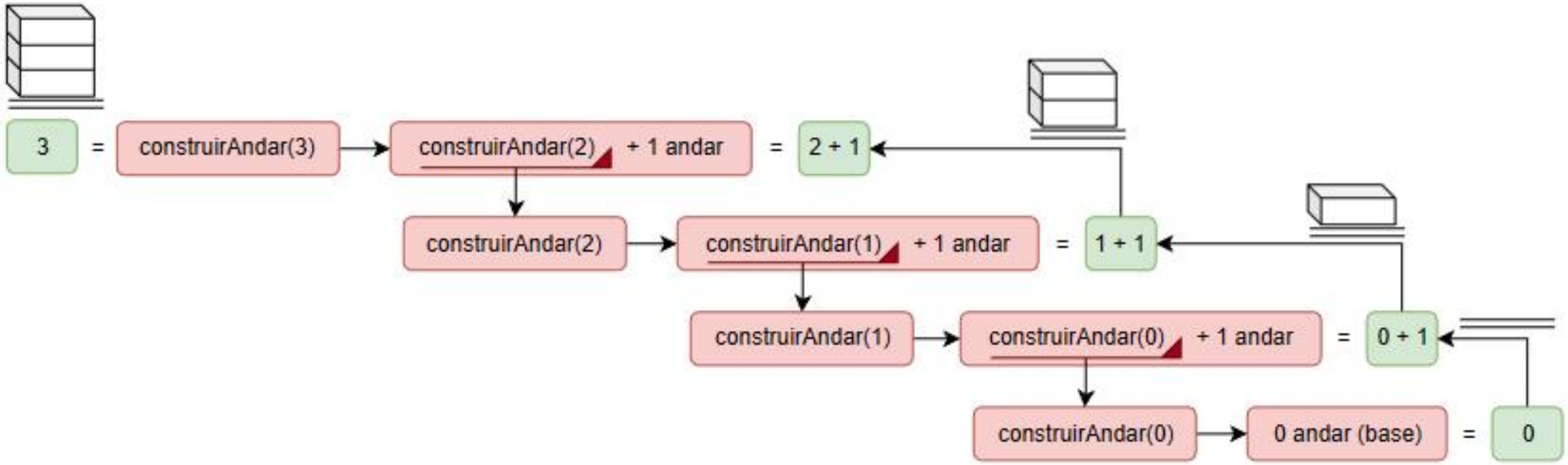
1. Construa o andar atual n, baseado na solução da construção de um prédio com n-1 andares.

2. Se o número do andar for igual a 0, pare.

Construção de um prédio de 3 andares

1. Construa o andar atual n, baseado na solução da construção de um prédio com n-1 andares.

2. Se o número do andar for igual a 0, pare.

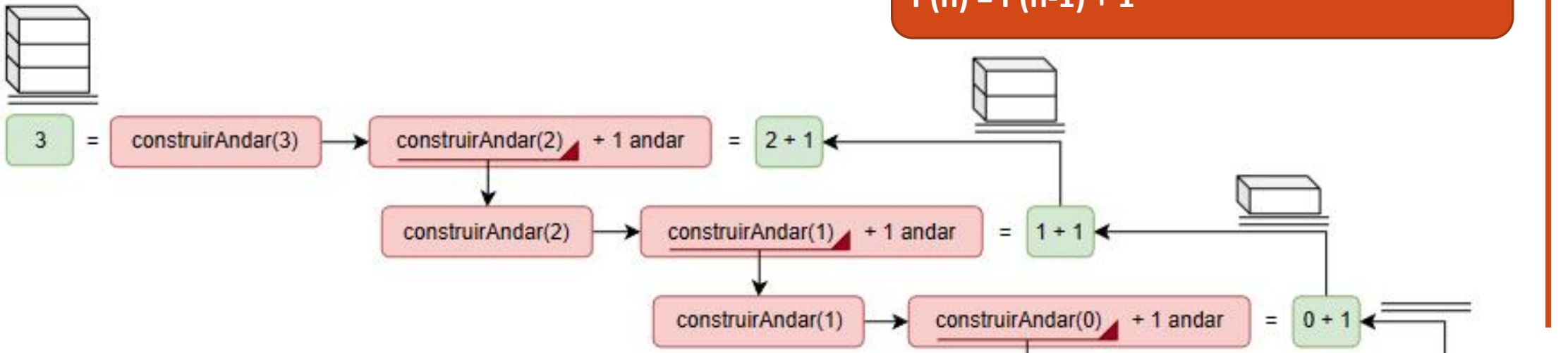


Construção de um prédio de 3 andares

1. Construa o andar atual n, baseado na solução da construção de um prédio com n-1 andares.

2. Se o número do andar for igual a 0, pare.

Para todo n > 0 (passo recursivo):
P(n) = P(n-1) + 1



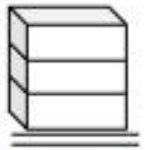
Critério de parada (caso base):
P(0) = 0

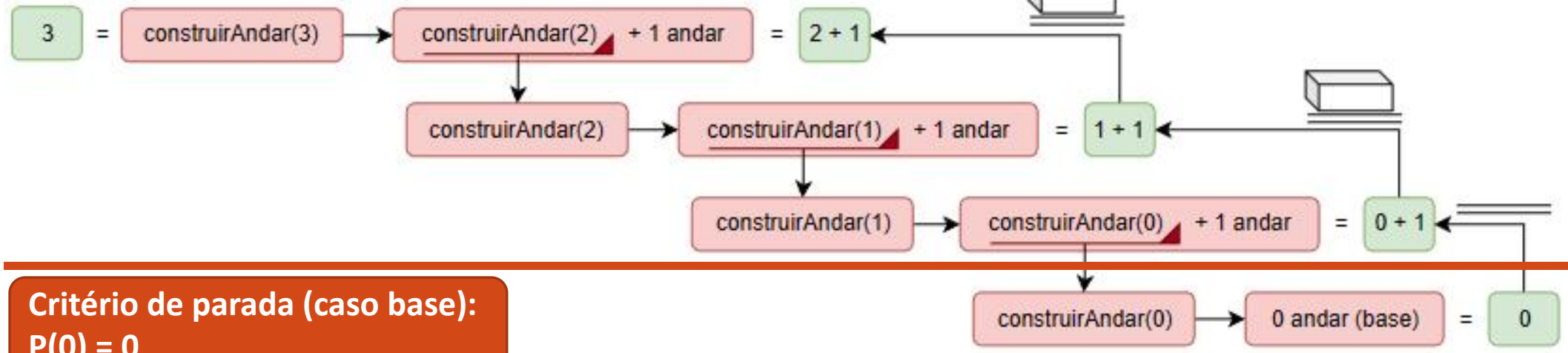
Definição Recursiva do problema do prédio

Para todo n >= 0:

$$p(n) = \begin{cases} 0 & \text{se } n = 0 \text{ (caso base)} \\ p(n-1) + 1 & \text{se } n > 0 \text{ (passo recursivo)} \end{cases}$$

Para todo n > 0 (passo recursivo):
P(n) = P(n-1) + 1





Critério de parada (caso base):
P(0) = 0

Algoritmo Recursivo

Para todo n >= 0:

$$p(n) = \begin{cases} 0 & \text{se } n = 0 \text{ (caso base)} \\ p(n-1) + 1 & \text{se } n > 0 \text{ (passo recursivo)} \end{cases}$$

```
// Algoritmo recursivo
int p(int n) {
    if(n > 0) return p(n-1) + 1;
    if(n == 0) return 0;
    // Exceções (n < 0) entrada inválida
    return 0;
}
```

Algoritmo Iterativo

```
// Algoritmo Iterativo
// Seja n=3,  (i <= 3)
//       (i<=0)  i------>i
//    p(n): 0   1   2   3
// Andares: 0 + 1 + 1 + 1
int pIterativo(int n) {
  int andares = 0;
  for(int i = 1; i <= n; i++) {
    andares = andares + 1;
  }
  return andares;
}
```

Algoritmos iterativos versus recursivos

Algoritmos recursivos:

A principal ***vantagem da recursão é a clareza e simplicidade do código***, pois permite expressar soluções de forma mais intuitiva e natural. Além disso, ela é útil para lidar com estruturas de dados complexas de forma recursiva.

Recursão pode ser ***menos eficiente que a iteração*** em alguns casos, pois cada chamada da função adiciona uma nova camada na pilha de execução, o que pode consumir mais memória

Algoritmos iterativos:

Extremamente difícil representar problemas mais complexos (não necessariamente impossível), porém não requer alocação de novas chamadas de função para resolver o problema, o que garante eficiência na execução do código.

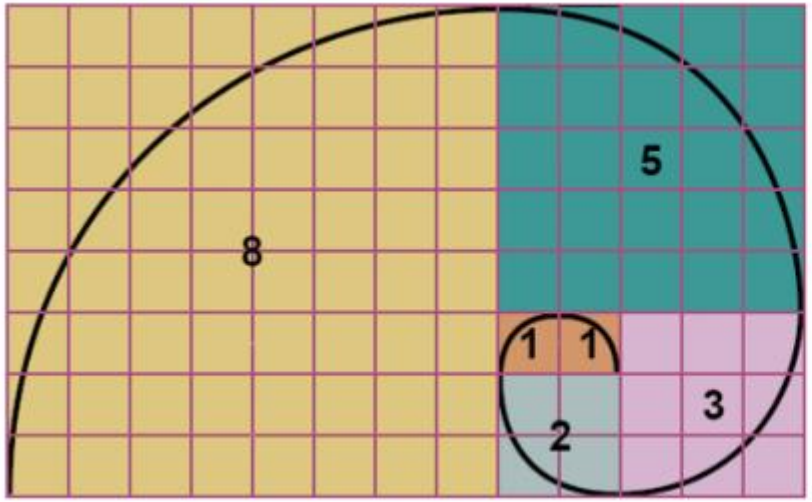
Recursão com Sequências de Números

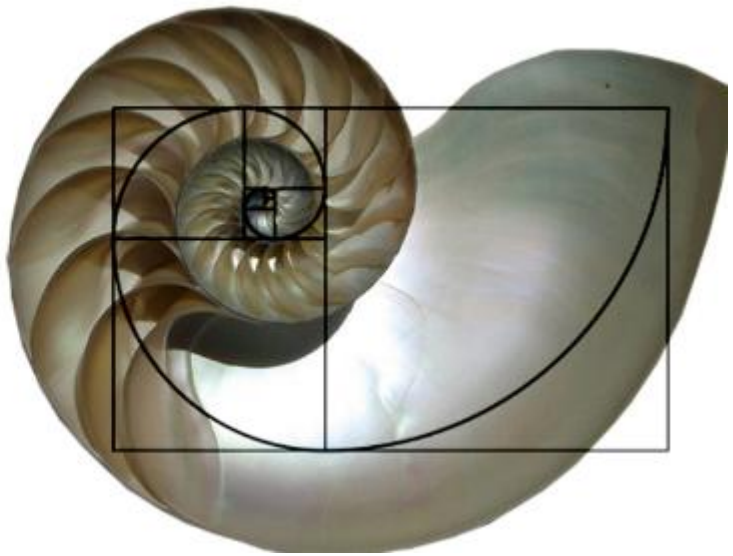
A recursão também pode ser utilizada para trabalhar com sequências de números.

Podemos descrever uma sequência recursivamente utilizando um ou mais casos base e um passo recursivo que especifica como cada termo é calculado a partir de termos anteriores.

Exemplo: Sequência de Fibonacci

1, 1, 2, 3, 5, 8, 13, 21, …





Exemplo: Sequência de Fibonacci

1, 1, 2, 3, 5, 8, 13, 21, ...

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...		
f(n)	Sequência	Relação de Recorrência
f(0)	0	- (critério de parada)
f(1)	1	- (critério de parada)
f(2)	1 = 0+1	fib(1)+fib(0)
f(3)	2 = 1+1	fib(2)+fib(1)
f(4)	3 = 2+1	fib(3)+fib(2)
f(5)	5 = 3+2	fib(4)+fib(3)
f(6)	8 = 5+3	fib(5)+fib(4)
...	...	...
f(n)	? = ?'+?"	fib(n-1)+fib(n-2)

se n = 0

se n = 1

para todo n > 1
(passo recursivo)

Definição Recursiva:

$$f(n) = \begin{cases} 0 & \text{se } n = 0 \text{ (caso base)} \\ 1 & \text{se } n = 1 \text{ (caso base)} \\ f(n-1) + f(n-2) & \text{se } n > 1 \text{ (passo recursivo)} \end{cases}$$

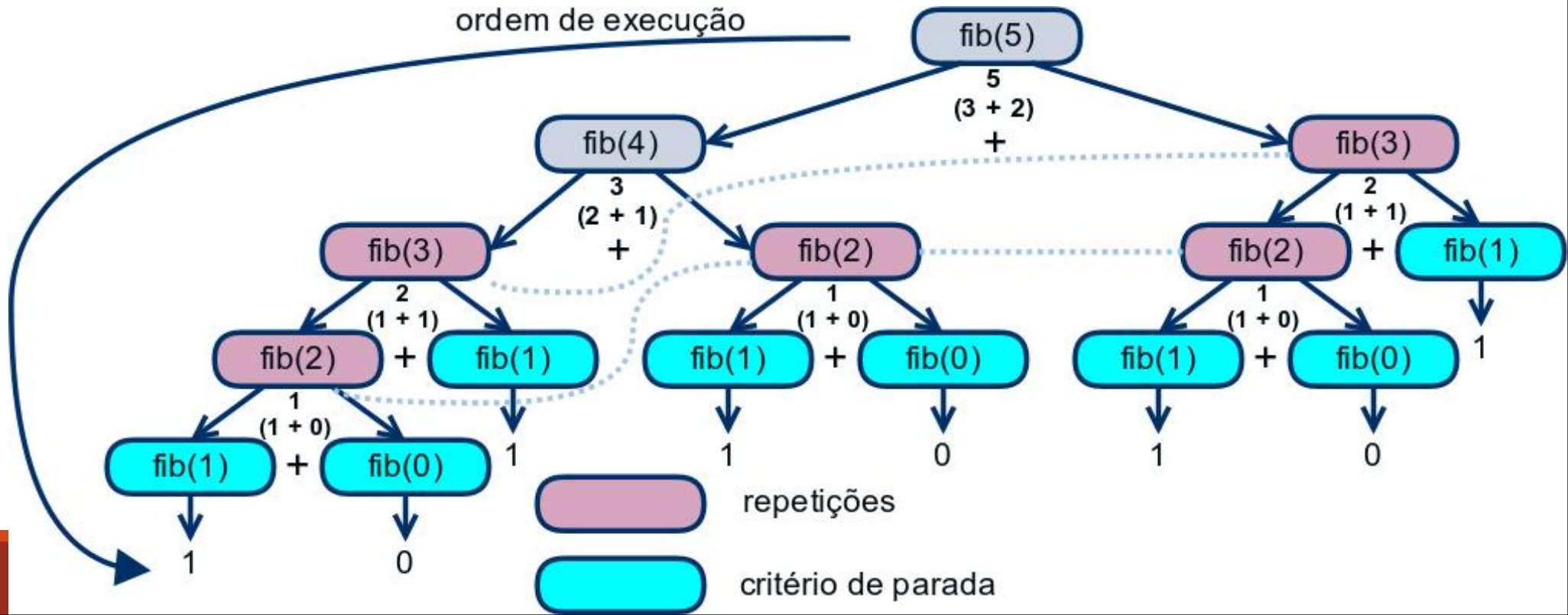
```
int f(int n) {
    if( n > 1) return f(n-1)+f(n-2); // passo recursivo
    if(n == 1) return 1; // critério de parada
    if(n == 0) return 0; // critério de parada
    // n < 0
    return 0; // exceção
}
```

Exemplo: Sequência de Fibonacci

1, 1, 2, 3, 5, 8, 13, 21, ...

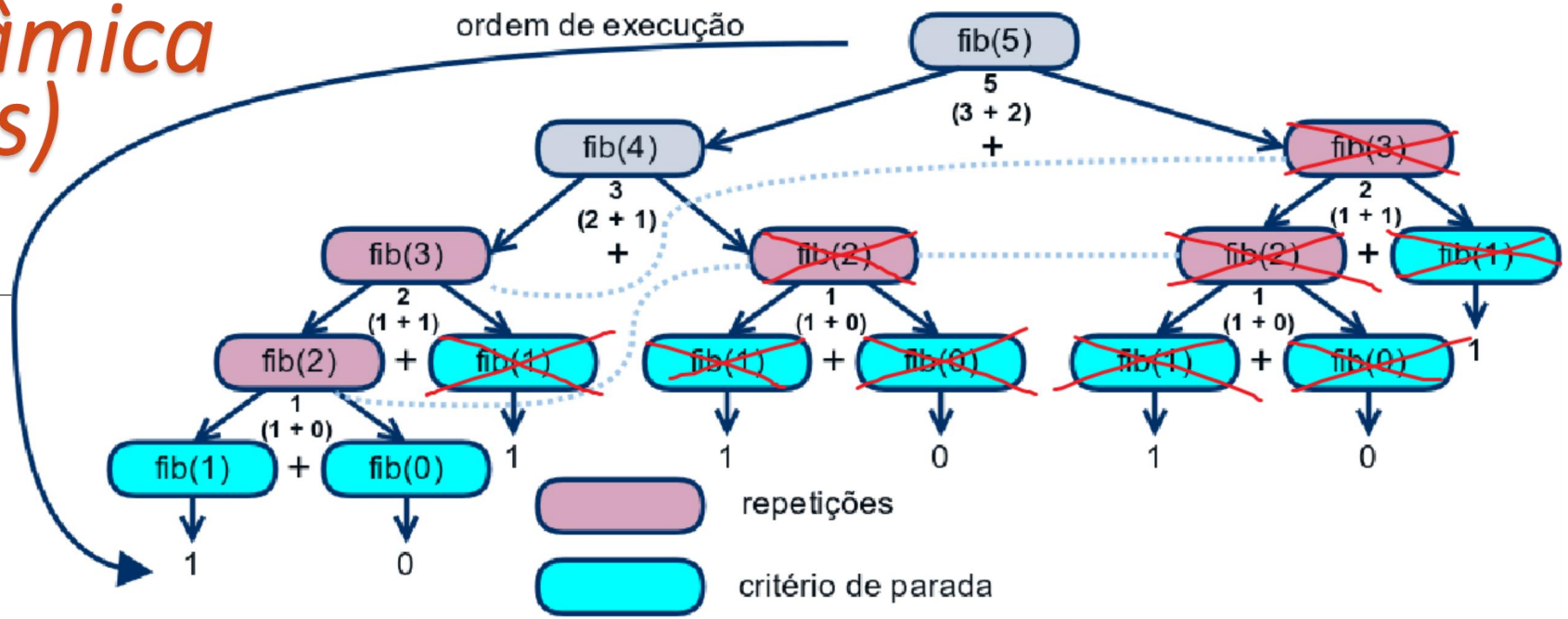
Definição Recursiva:

$$f(n) = \begin{cases} 0 & \text{se } n = 0 \text{ (caso base)} \\ 1 & \text{se } n = 1 \text{ (caso base)} \\ f(n-1) + f(n-2) & \text{se } n > 1 \text{ (passo recursivo)} \end{cases}$$



Programação dinâmica (elimina repetições)

1, 1, 2, 3, 5, 8, 13, 21, ...




```
//Programação Dinâmica
int fibs[1000] = {0}; // vetor inicializado com 0
int f(int n) {
    if(fibs[n]) return fibs[n];
    if( n > 1) { // passo recursivo
        if(!fibs[n-1]) fibs[n-1] = f(n-1);
        if(!fibs[n-2]) fibs[n-2] = f(n-2);
        return fibs[n-1]+fibs[n-2];
    }
    if(n == 1) return 1; // critério de parada
    if(n == 0) return 0; // critério de parada
    // n < 0
    return 0; // exceção
}
```

Exercício

Construa a definição recursiva para o seguinte produtório:

$$\prod_{i=1}^{n} i = 1 \times 2 \times 3 \times \cdots \times (n-2) \times (n-1) \times n$$

Resolução

$$\prod_{i=1}^{n} i = 1 \times 2 \times 3 \times \cdots \times (n-2) \times (n-1) \times n$$

$$\prod_{i=1}^{n} i = 1 \times 2 \times 3 \times \cdots \times (n-2) \times (n-1) \times n$$

f(n)	Sequência	Relação de Recorrência
f(1)	1	- (critério de parada)
f(2)	$2 = 1 \times 2$	$f(1) \times 2$
f(3)	$6 = 1 \times 2 \times 3$	$f(2) \times 3$
f(4)	$24 = 1 \times 2 \times 3 \times 4$	$f(3) \times 4$
...	...	...
f(n)	$? = 1 \times 2 \times 3 \times 4 \cdots \times (n-2) \times (n-1) \times n$	$f(n-1) \times n$

f(1)

f(2) = f(1) x 2

f(3) = f(2) x 3

f(4) = f(3) x 4

...

f(n-2) = f(n-3) x (n-2)

f(n-1) = f(n-2) x (n-1)

f(n) = f(n-1) x n

se n = 1

para todo n > 1
(passo recursivo)

Definição Recursiva:

$$f(n) = \begin{cases} 1 & \text{se } n = 1 \text{ (caso base)} \\ f(n-1) \times n & \text{se } n > 1 \text{ (passo recursivo)} \end{cases}$$

```
int f(int n) {
    if(n  > 1) return f(n-1)*n; // passo recursivo
    if(n == 1) return 1; // critério de parada
    // n < 1
    return 0; // exceção
}
```

Exercício

Construa a definição recursiva para a seguinte sequência:

2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, ...

Resolução

2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, ...

2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, ...		
f(n)	Sequência	Relação de Recorrência
f(1)	2	- (critério de parada)
f(2)	$4 = 2 \times 2$	$f(1) \times 2$
f(3)	$8 = 4 \times 2$	$f(2) \times 2$
f(4)	$16 = 8 \times 2$	$f(3) \times 2$
f(5)	$32 = 16 \times 2$	$f(4) \times 2$
f(6)	$64 = 32 \times 2$	$f(5) \times 2$
...	...	...
f(n)	$? = ?' \times 2$	$f(n-1) \times 2$

se n = 1

para todo n > 1
(passo recursivo)

Definição Recursiva:

$$f(n) = \begin{cases} 1 & \text{se } n = 1 \text{ (caso base)} \\ f(n-1) \times 2 & \text{se } n > 1 \text{ (passo recursivo)} \end{cases}$$

```
int f(int n) {
    if(n > 1) return f(n-1)*2; // (passo recursivo)
    if(n == 1) return 2; // (critério de parada)
    return 0; // n < 1: exceção
}
```